

swtpcsim — The Monitor

2026-09-27 · 0e4d89e

*A simulation of the SWTPC 6800 on the SS-50/SS-30 bus, and the MITS Altair
680b*

The monitor	1
Commands resolve by prefix	1
Editing the command line	2
Repeating the last command:	3
Reaching the host: !	4
Numbers: one rule, and it is not negotiable	4
Naming a board: <id>[:<unit>]	6
Seeing the machine	6
Changing the machine	7
Running, and stopping	7
Speed	8
RESET is not POWER	9
Every monitor command	10
Running the machine	10
Examining and changing memory	13
Debugging and tracing	17
Configuring the machine	21
Getting help and leaving	31

The monitor

This is one of two short documents about **driving swtpcsim itself** — the program you run, as opposed to the machines it emulates. It covers **the monitor**: the `swtpcsim>` prompt where you start, stop, examine and reconfigure the machine. Its companion, *The Debugger*, covers what you reach for at that prompt when something has gone wrong. Both stand beside the *User Manual*, which describes the hardware — the boards, the machines, the disks and the tapes. `swtpcsim` simulates the SWTPC 6800 and the MITS Altair 680b — Motorola 6800 machines on the SS-50/SS-30 bus; if that is new to you, the manual's opening chapter is the place to start.

The `swtpcsim>` prompt is **the monitor**: the machine's operator's position, and its debugger, which here are the same thing. On the Altair 680b that position was a real front panel of switches and lamps; on the SWTPC 6800 it was a terminal at SWTBUG's \$. Everything those gave you the monitor can do — and a great deal they could not. It breakpoints, it single-steps, it disassembles, and it will show you the bus itself: who decodes what, who is pulling which interrupt line, and where two boards are fighting over an address. That is what *The Debugger* is for, and it is most of why this program exists.

What it is not is a *menu* — a layer sitting between you and the machine, offering a fixed set of things it is prepared to let you inspect. There is no debug mode to enter and nothing is watching from the outside. A breakpoint is **the machine stopping**, not a script noticing that it should have. Because the 6800 is **memory-mapped** — its devices live in the address space, not a separate port space — `EXAMINE` and `DEPOSIT` run **real bus cycles**, with every side effect a real one has: read the console ACIA's data register at `8005` from this prompt and you have taken the byte, exactly as the guest would have. The operator's position and the debugger are one object because on a 680b they were one object: a hand at the switches, reading lamps.

The machine does not have to be running for the monitor to work. Most of what follows — examining memory, running a bus cycle, fitting a board — works on a machine with the power on and the processor idle, which is exactly the arrangement the 680b's front panel was for.

Commands resolve by prefix

There are **no aliases and no memorised abbreviations**. You type as much of a command as it takes to be unambiguous, and the first command that matches wins.

`HELP` prints the whole menu with each command's shortest form in brackets:

BO[ARDS]	B[REAK]	COM[PARE]	C[ONFIG]
CONN[ECT]	CONS[OLE]	DE[POSIT]	DI[SASM]
DISC[ONNECT]	DO	D[UMP]	E[DIT]
EX[AMINE]	F[ILL]	HE[LP]	H[ISTORY]
I[N]	L[OAD]	MA[CHINE]	M[OUNT]
MOV[E]	N[EXT]	NO[BREAK]	O[UT]
P[OWER]	Q[UIT]	REGI[ON]	RE[GS]
RES[ET]	REST[ORE]	R[UN]	SA[VE]
SEA[RCH]	SE[T]	SH[OW]	SN[APSHOT]
STA[RTUP]	S[TEP]	SY[MBOLS]	T[RACE]
TY[PE]	U[NMOUNT]	W[HO]	

Type the part before the bracket. `D` is `DUMP`; `DE` is `DEPOSIT`; `RES` is `RESET`. Case does not matter, here or in the name of any board.

`HELP <command>` gives you the usage and worked examples for one of them, and `?` is the same as `HELP`.

R is RUN, not RESET. It is the command you type every session, and it is the one that costs nothing if you did not mean it. A bare `R` that reset the machine would be one you had to set up again. `RESET` pays the letters: `RES`.

Editing the command line

The prompt is a real line editor. The key labelled Backspace erases the character behind the cursor whichever byte your terminal sends for it, the arrows move within the line, and the editor keeps a **command history** — the up-arrow walks back through the lines you have typed and the down-arrow returns toward the one you were in the middle of.

The history is **saved between sessions, per directory**. When you leave, the last commands you typed are written to a hidden `.swtpcsim_history` in the directory you launched from, so the next time you start the simulator *there* they are waiting on the up-arrow. Each project directory keeps its own list; the file is written only when you are typing at a real terminal, so a script, a pipe, or an automated run never leaves one behind. How many lines it keeps is the `history` setting on the console — `SET CONSOLE history=200` to keep more, and `SET CONSOLE history=0` to turn the file off entirely. It defaults to 50.

Completing with `Tab`

`Tab` finishes whatever you are partway through typing, and it reads the candidates off the machine in front of you — so a board you plug in is completable straight away, with nothing to keep up to date:

- at the start of a line, a **command** — `SH` → `SHOW` ;
- after `SET` , a **board id** (or `CONSOLE` , `DISPLAY`) — `SET me` → `SET mem0` ;
- after a board, one of **its property names**, with the `=` put on ready for the value — `SET mem0 fi` → `SET mem0 fill=` ;
- after the `=` , one of that property's **legal values** — `SET mem0 fill=` → offers `zero` and `random` .

When more than one candidate fits, `Tab` fills in as far as they all agree and stops; press it again and it lists them. When nothing fits, it does nothing.

The keys

Key	Does
<code>←</code> <code>→</code>	move one character
<code>Ctrl-A</code> / <code>Home</code>	to the start of the line
<code>Ctrl-E</code> / <code>End</code>	to the end of the line
<code>Alt-B</code> / <code>Ctrl-←</code>	back one word
<code>Alt-F</code> / <code>Ctrl-→</code>	forward one word
<code>Backspace</code>	erase the character before the cursor
<code>Delete</code>	erase the character under the cursor
<code>Ctrl-W</code>	erase the word before the cursor
<code>Ctrl-K</code>	erase from the cursor to the end of the line
<code>Ctrl-U</code>	erase the whole line
<code>↑</code> <code>↓</code>	walk back and forth through the command history
<code>Tab</code>	complete the word at the cursor
<code>Ctrl-D</code>	on an empty line, leave — the same as <code>QUIT</code>

Ctrl-E here is end-of-line, not the STOP key. At the prompt you are typing to the *editor*, so `Ctrl-E` jumps to the end of the line. Once a **running** guest holds the console, that same `Ctrl-E` is **STOP** and takes the keyboard back (below). Same key, two places, two jobs.

Repeating the last command: `.`

Type `.` on a line by itself and the monitor runs your **last command again**, quietly — there is no echo, just the command's own output. It costs one keystroke, and the commands you most often want to repeat

pick up where they left off: a bare `DISASM` disassembles the next screenful, a bare `DUMP` shows the next page, and `STEP` steps again. So you type `DI` once and then `. . .` to walk forward through a routine, or `S` and then `.` to single-step.

Pressing `.` again always repeats that same original command, never the previous `.`, so it keeps doing the one thing however many times you press it. A `.` before you have typed anything just tells you there is nothing to repeat yet.

Reaching the host: `!`

A line that begins with `!` is not a monitor command at all — everything after the `!` is handed to **your host shell**, word for word, and the monitor waits until it is done before it prompts again. The rest of the line is passed through untouched, spaces and all:

```
!ls                list the directory you started from
!vi HELLO.PRN      open a file in your editor, then :q back to the prompt
!cp game.dsk save.dsk  keep a copy of a disk without unmounting it
```

This is **your** shell, with your own privileges — not the machine's, and nothing the guest can see or reach. It is also why an editor works: the monitor is not holding the keyboard when it hands off, so `vi` gets a normal terminal and gives it back when it exits. The machine keeps running underneath; `!` only borrows *you*, not the processor.

A bare `!` with nothing after it just reminds you of the form.

Numbers: one rule, and it is not negotiable

On the wire → hex. Never on the wire → decimal.

If the 6800 can see it, it is **hex**: an address, a device register, a data byte, a CPU register. If it never leaves your head, it is **decimal**: a count, a width, a size, a drive number.

Hex is only the *default* for the wire class. Switch the console to octal and that class reads and prints in octal instead — the base the 680b's monitor and MITS's Motorola manuals used as readily as hex. The rule

does not change: octal is still the wire class, decimal is still the counts. *Reading and writing in octal*, below, is how.

```
DUMP 100          address -> 0100 hex
STEP 10           a count  -> ten instructions
DEPOSIT 8010 55    address and byte -> both hex
SET mps0:tty baud=9600    a baud rate -> nine thousand six hundred
```

You can always force the issue: `0x`, `$` and a trailing `h` force hex; `0o` and a trailing `q` force octal; `0b` forces binary — which is what you want for a bit-mapped value like the 680b's config straps, where eight bits would rather be eight digits; a leading `#` forces decimal; and a `K` or `M` suffix is **always** decimal (`48K` is 49,152 — so `0x10K` is a contradiction and is rejected rather than guessed at).

This rule is the same everywhere — in the monitor, in a machine file, and in every board's settings. There is no second convention to learn.

What is not negotiable is the **classes** — which side of the line a number falls on. The base the wire class is *printed* in is yours, and the next section is how.

Reading and writing in octal

The 680b's programming manual and MIT's Motorola literature wrote numbers in **octal** as readily as hex, and you can too:

```
SET CONSOLE base=octal
```

Now the **hex** half of the rule becomes **octal** — the wire class (addresses, data bytes, CPU registers) is read and printed in **split octal**, each byte its own `000 – 377` group and a 16-bit address as two of them, the way a 680b front panel groups its address lamps:

```
EXAMINE 100      -> 000 100 076    (the byte 0x3E at address 0x40)
DUMP 100-100     -> 000 100 076
DISASM 0         -> JMP 022 064     (a jump to 0x1234)
```

A bare number is octal now too, so `100` is address `0x40` ; the decimal class (counts, widths, baud) does not change. The forcing markers still work in both directions — `0x1234` is hex even in octal mode, and `0o377` is octal even in hex mode — so nothing is ever a base you cannot type your way out of. `base=hex` (the default) puts it back. Set it once in a machine file (`[console] base = octal`) to start there every time.

Naming a board: `<id>[:<unit>]`

Every board in the machine has an **id** you chose (`cpu0` , `mps0` , `dc40`), and some boards have **units** inside them — the console line on a serial board, the four drives on a floppy controller, the ROM sockets on a memory board.

```
SHOW mps0           the board
SET  mps0:tty baud=1200 its console line
MOUNT dc40:drive1 my.dsk
```

You may leave out anything that carries no information. If there is only one floppy controller in the machine, `dc4` will find it. If a board has only one thing you could mount into, you need not name it — `MOUNT kc0 tape.tap` puts a cassette in the one recorder.

But anything **genuinely plural you must say**. There are four drives on that controller and the machine will not guess which one you meant; it will tell you so and stop.

Seeing the machine

```
swt pcsim> BOARDS
```

ID	TYPE	UNITS	MEMORY
cpu0	6800	1 cpu: 6800	-
mps0	mps	1 serial: tty*	8004-8007 6850 ACIA 'tty'
dc40	dc4	4 disk: drive0(empty), drive1(empty), ...	8014-8014 drive select 8018-801B WD179x
mem0	memory	2 rom: rom0, rom1	0000-7FFF ram 32K A000-BFFF ram 8K E000-E3FF rom swtbug FC00-FFFF rom swtbug

* holds the console

That is the backplane: what is plugged in, what each board decodes on the bus, what is in its units, and where it answers in memory. (The 6800 is memory-mapped, so the `I/O` column — a separate port space — stays empty; every device shows up under `MEMORY`.)

Command	Shows
<code>BOARDS</code>	the backplane
<code>SHOW <id></code>	one board: every setting, its value, and what it will accept
<code>SHOW MACHINE</code>	the whole machine
<code>SHOW CONSOLE</code>	which unit holds your keyboard, and how bytes are being transformed
<code>SHOW DISPLAY</code>	the video window: whether it or the terminal has the keyboard, and whether it wears the CRT look
<code>SHOW BUS MAP</code>	who decodes which addresses — and what floats
<code>SHOW BUS IRQ</code>	the IRQ and NMI wires, who is pulling them, and the vectors at <code>FFF8 – FFFF</code>
<code>SHOW BUS CONTENTION</code>	where two boards are fighting

`SHOW <id>` is worth dwelling on, because it is the **only** thing you need in order to configure a board. It lists every property, what it is set to, and what values are legal — and those property names **are** the keys you write in a machine file. There is no second schema anywhere in this program. The board reference at the back of this manual is printed from the same source.

Changing the machine

```
SET cpu0 clock_hz=1000000    give it the real 1 MHz crystal
SET mem0 fill=zero           RAM comes up zeroed instead of random
SET mps0:tty baud=1200       slow the console down
BOARDS ADD mps mps1 base=8008 fit a second serial board
BOARDS REMOVE mps1           pull it out
CONFIG SAVE mine.toml        write out the machine you are actually running
```

`CONFIG SAVE` round-trips: what it writes, `swtpcsim mine.toml` will boot.

Running, and stopping

```
RUN E0D0    load the PC and go – the same two motions as the 680b panel's switches
RUN         carry on from wherever the processor is
```

RUN <addr> is EXAMINE followed by RUN, exactly as you would do it on the front panel. There is no **B00T** command in this program, and there should not be: a machine that ought to start says so with the operator's own keystroke.

If a board holds the console, **the guest gets the keyboard** — every key, including **^C**, which a FLEX program is entitled to read.

STOP takes it back

^E presses the front-panel STOP switch. The host intercepts it *before the guest is ever offered the byte*, so no program running inside the machine can disable it, trap it, or take it from you.

```
$  
STOP -- the machine is still at E201. RUN resumes.  
swtpcsim>
```

STOP halts the machine and gives you the monitor. Nothing executes while this prompt is up. But it stops the machine without *disturbing* it — STOP is not RESET and not POWER, so the registers, the memory and the disk are exactly as the guest left them, and a bare **RUN** (no address) picks up at the very instruction it was about to execute. That is what "*still at E201*" is telling you.

CONSOLE stop=1D moves STOP to **^J** if **^E** collides with something the guest wants (the older **attn=** spelling still works).

What stops it for real

A **RUN** ends when it hits a **breakpoint**, or a **HLT** that nothing can wake — and it always says which. With no console connected there is nothing to hand the keyboard to, so it simply runs, and **^C** stops it.

RUN and ^E are the panel's RUN and STOP switches. **RUN** starts the processor; STOP — or a breakpoint, or a **HLT** — stops it and hands the monitor back. That prompt is the whole distinction: **the monitor exists only while the machine is stopped**. While it runs, the guest holds the keyboard and there is no **swtpcsim>** to type at; when you have the prompt, nothing is executing. So every **SET**, **DEPOSIT** and **EXAMINE** you type acts on a *stopped* machine — a property is never changed out from under a running instruction, and none is ever locked "while running" or settable only then.

Speed

It runs flat out by default — **clock_hz** on the CPU board is **0**, so a cassette that took a real 680b a couple of minutes comes off in about a second. **SET cpu0 clock_hz=1000000** buys back the 1 MHz SWTPC machine (**500000** for a 680b); what the guest sees is identical either way, because the tape still

costs the same cycle count — the crystal buys period *feel*, not *behaviour*. `SHOW cpu0` reports `achieved_hz` beside it: the clock the run loop actually hit, a measurement you cannot set.

The one exception is anything the guest times against the *outside* world — an XMODEM transfer wants the real crystal, a cassette does not. The manual's Boards chapter (`clock_hz` , `idle`) and its Troubleshooting chapter have the detail.

RESET is not POWER

RESET	the bus's RESET* line. The processor loads its PC from the RESET vector at <code>FFFE / FFFF</code> (→ the monitor's entry). Memory survives , disks stay mounted.
POWER	a power cycle. This is the only thing that loses RAM and re-reads the ROM images.

The 6800 does not restart at `0000` : reset arms the two-byte fetch at the top of memory and the first instruction it runs comes from wherever `FFFE / FFFF` point — SWTBUG's RESET entry at `E0D0` on the SWTPC, MON680's on the 680b. `RESET` does not clear memory, because pressing RESET on a real 6800 machine did not clear memory — that is behaviour a lot of period software depends on.

`RESET*` is a **line on the backplane**, not an instruction the simulator carries out for you, and every board hears it and answers the way its own silicon did — which is not the same answer twice. The memory board touches no RAM (a RAM chip has no reset pin); the floppy controller flushes the sector it was writing and deselects the drive; and the console board's 6850 does **nothing at all**, because the 6850 has no reset pin for `RESET*` to land on — so its baud rate, word format and interrupt enables all survive a reset, exactly as on the bench. Hit `RESET` mid-write and you get what the hardware gave you: a half-written sector, a serial port still configured as the dead program left it, and every byte of RAM intact.

POWER is a different wire. Switching the machine on drives `POC*` — Power-On Clear, its own backplane line — and a board may treat the two differently, because the real cards did. The console 6850 is the case that proves it: `RESET*` has no pin to land on, so a baud rate, word format and interrupt-enable a crashed program left set **survive a RESET** and only return to their power-on state on `POWER` . `POC*` is also the only moment RAM is allowed to forget: on `POWER` the memory board refills itself — with **random bytes by default**, because static RAM does not come up zeroed — and re-reads every ROM image.

	The processor	The boards	RAM
RESET	loads PC from <code>FFFE / FFFF</code>	<code>RESET*</code> on the bus; each board answers as its silicon did — some do nothing	survives
POWER	loads PC from <code>FFFE / FFFF</code>	<code>POC*</code> on the bus; the boards come up as they do from cold	refilled , ROMs re-read

Every monitor command

Commands resolve by prefix, and the first match wins. There are no aliases and no fixed abbreviations: the shortest prefix that reaches a command is derived from the table's order, so it is shown here as `D[UMP]` — type the part before the bracket. (`?` is the one true alias, for `HELP`.)

. repeats your last command. It runs quietly, with no echo, so a single keystroke walks the continuing verbs forward: `DI` disassembles a screenful, then `.` `.` `.` keeps going; the same holds for `DUMP` and `STEP`.

Numbers: on the wire is **hex** (addresses, ports, bytes); never on the wire is **decimal** (counts, widths, sizes). `0x` / `$` / `h` force hex, `0o` / trailing-`q` force octal, `#` forces decimal, and a `K` / `M` suffix is always decimal. `SET CONSOLE base=octal` makes the wire class read and print in **split octal** (each byte its own `000 – 377` group, an address as two of them) — a period front-panel convention; `base=hex` is the default. Either way both spellings stay typeable.

The commands are **grouped by what they do**, and within a group they are in **alphabetical order**. The abbreviation beside each name is still derived from the master table's priority order, not from this listing, so it is what you type regardless of where the command sits here.

Running the machine

NEXT — `N[EXT]`

NEXT

STEP that does not descend. A JSR or BSR runs to completion and stops at the return address instead of stepping into it; anything else is a plain single step. It is a temporary breakpoint at the return plus a RUN, so the callee is LIVE -- it can use the console, and `^E` (STOP) or `^C` stops it.

N over the JSR/BSR at PC (else single-step)

POWER — `P[OWER]`

POWER

Power cycle. THE ONLY THING THAT LOSES RAM -- a RESET does not, because on real hardware it does not.

RESET — RES[ET]

```
RESET [CPU]
```

A reset does NOT clear memory. Only removing power does that -- see POWER. RESET CPU is a debugging convenience, NOT a real signal: no wire on the backplane resets the processor and nothing else.

RUN — R[UN]

```
RUN [addr]
```

Start the machine. `RUN <addr>` is EXAMINE + RUN -- it loads the PC first, exactly as you would on the panel.

If a unit holds the console, the GUEST GETS THE KEYBOARD -- every key, including ^C, which the guest software is entitled to read. The way back is STOP (^E), which the host takes before the guest is ever offered the byte, so the guest cannot disable it. STOP halts the machine -- nothing executes while this prompt is up -- but it does not DISTURB it: it is not RESET and not POWER, so every register, every byte and every disk survives, and a bare RUN resumes at the exact instruction. Stopped is not lost, and the debugger is at its most useful here: REGS, EXAMINE, DUMP, DISASM and STEP all work at this prompt.

IT RUNS FLAT OUT unless the CPU board has a crystal. `clock_hz` defaults to 0, so a cassette that took a real 680b a couple of minutes comes off in about a second. `SET cpu0 clock_hz=1000000` buys back the 1 MHz machine AND its couple of minutes. What the guest sees is identical either way -- the tape still costs the same clock cycles -- so the crystal buys period FEEL, not period behaviour.

With no console connected there is no keyboard to hand over, and nothing to pace against: it simply runs, ^C stops it. Either way it stops on a breakpoint or on a WAI nothing can wake, and it ALWAYS says which.

```
RUN E000      start SWTBUG in ROM
RUN           carry on from wherever the PC is
```

STEP — S[TEP]

```
STEP [n]
```

One instruction, with REAL bus cycles through the real decode. Prints one line per instruction; past 32 it runs quietly and reports. `n` is a count, so it is decimal.

A LINE IS THE STATE AFTER THE INSTRUCTION RAN -- the registers as they now stand, and the instruction the PC has reached next. So `S 3` prints three lines, one per step, and the last line is where the monitor has left you: the next instruction, not yet run.

```
S          one instruction
S 10       ten of them
```

```
swtpcsim> DEPOSIT 0 86 08 C6 0A 1B 3E          LDAA #08 / LDAB #0A / ABA / WAI
swtpcsim> EX 0
swtpcsim> S 3
H0I0N0Z0V0C0 A=08 B=00 X=0000 SP=0000 PC=0002 LDAB #0A
H0I0N0Z0V0C0 A=08 B=0A X=0000 SP=0000 PC=0004 ABA
H1I0N0Z0V0C0 A=12 B=0A X=0000 SP=0000 PC=0005 WAI
```

Three instructions ran, three lines. Each shows the result: A=08 lands on the line for the LDAA that loaded it, and the last line is the WAI waiting, not yet run. The flags are the 6800's own six, in the hardware lamp order H I N Z V C -- Half-carry, Interrupt mask, Negative, Zero, oVerflow, Carry -- and H goes to 1 on the ABA because 8+A carries out of bit 3.

TYPE — TY[PE]

```
TYPE "text"
```

Put characters in the console's input buffer as if they were typed at the guest -- the same keystrokes the VDM window or a terminal would send. The guest reads them when it next looks at the keyboard, so they are TYPE-AHEAD: they wait in the buffer and are not forced on a program that is not reading.

Escapes: `\r` carriage return, `\n` line feed, `\t` tab, `\` backslash, `"` quote. ENTER sends a carriage return, so a command line for the guest ends in `\r`.

This is how a machine file feeds a guest program its first command. `startup` runs MONITOR commands; `CAT` is input to FLEX, a program INSIDE the machine. Put TYPE before the RUN that starts the guest and the guest reads it at its first prompt -- in a TOML the backslash doubles (`\r`), because the config parser keeps one for the command:

```
startup = ["MOUNT dc40:drive0 \"FLEX2-40.DSK\"", "TYPE \"CAT\\r\"", "RUN E000"]
TYPE "CAT\r"          type it now, at a running guest
```

A program that clears its keyboard as it starts drops keystrokes sent before it is ready; TYPE cannot help there, no more than a fast typist could.

Examining and changing memory

COMPARE — COM[PARE]

```
COMPARE <range> <addr>
```

Byte for byte, a against the SAME LENGTH starting at -- memory to memory, both in the machine's address space. Every mismatch prints both addresses and their bytes; then a total. It changes nothing and runs no cycle.

```
COMPARE 0-FF 200      page 0 against page 2
COMPARE FF00-FFFF E000 the boot PROM against a copy up at E000
```

DEPOSIT — DE[POSIT]

```
DEPOSIT <addr> <bytes...>
```

The front-panel switch. Runs a REAL bus write, so if no board decodes the address the byte is simply gone -- and DEPOSIT says so rather than lying.

```
DE 100 C3 00 F8
```

DUMP — D[UMP]

```
DUMP [<addr>|<range>] [WIDTH=16]
```

Hex and ASCII. A bare address runs to the END OF ITS PAGE, and a bare DUMP continues from there -- so the rows and the columns both stay page-aligned however you first landed. WIDTH is a count, so it is decimal.

```
D 100      0100-01FF, a whole page
D 0001     0001-00FF: stops on the boundary, last line full
D          the next page
D FF00-FF0F an explicit range means exactly what it says
D 100/20    0100-011F (LEN is part of the address expression: hex)
D 0 WIDTH=8 eight bytes per line
```

EDIT — E[**DIT**]

```
EDIT <addr> [ROM]
```

Interactive DEPOSIT. The prompt shows an address and the byte that is there; type a new value and Enter writes it and drops to the next byte, bare Enter leaves it and drops to the next, and '.' returns to the monitor. Runs REAL bus writes, so it says so if no board decodes the address; ROM burns instead, the way LOAD ... ROM does -- behind the bus, into the chip that answers there. If the machine has a CPU, type an INSTRUCTION where a byte would go and it is assembled in place -- the prompt then drops by the instruction's length, not one byte. Operands are numbers in the console base (an H or Q suffix overrides); a bare value is still a plain byte. The 6800 assembles in full -- immediate, direct, indexed and extended addressing, and the branches as a target the assembler turns into a signed offset. The 6809 does too, and adds every indexed form (n,X [n,X] ,X++ A,X n,PCR [nnnn]), TFR/EXG pairs and PSH/PUL lists. On a 6809, nnnn is extended. Look at a few bytes, patch two instructions in, and read them back:

```
swtpcsim> EDIT 100
0100 00 LDAA #08      assembles 86 08, on to 0102
0102 00 STAA 8004      assembles B7 80 04, on to 0105
0105 00 .             '.' returns to the monitor
swtpcsim> DISASM 100 2
0100 86 08      LDAA #08
0102 B7 80 04  STAA 8004
(needs an interactive or piped session -- with none, use DEPOSIT)
```

EXAMINE — EX[**AMINE**]

```
EXAMINE [<addr>]
```

One byte: hex, ASCII, and the bits as the panel's LEDs showed them. EXAMINE jams the address into the PROGRAM COUNTER, just like the panel switch, so EX <addr> also shows the register line and the disassembled instruction the PC now points at -- what the next STEP will execute. Bare EXAMINE is the panel's EXAMINE NEXT -- it steps one byte, quietly. Its cursor is its own; a DUMP does not move it.

```
EX 100      0100 C3 . 11000011
            A=00 ... PC=0100 JMP 0138
EX          and the next byte, and the next
```

FILL — F[**ILL**]

```
FILL <range> <byte>
```

FILL 0-3FF 00

LOAD — L[OAD]

LOAD <file> [AT <addr>] [FORMAT=BIN|HEX|SREC] [ROM]

Put a file into memory. There are three kinds of file and they differ in ONE way -- whether the file knows where it goes.

```
HEX Intel HEX. ASCII text, and it CARRIES ITS OWN ADDRESSES, so it needs
no AT. Each line is one record: a ':', a length, the address, a type
(00 data, 01 end-of-file), the bytes, and a checksum. Every checksum
is verified and a bad one FAILS the load and names the record -- a
half-loaded program is a miserable thing to debug.

:10010000C300F8AF32004D3E0132014D76C9AA5509
:00000001FF
^^^^^^^^^          ^^ checksum: the record
| | |              sums to zero, byte-wise
| |   type 00 = data (01 = end of file)
| load address: these bytes go at 0100
10 = sixteen data bytes follow
```

```
SREC Motorola S-records -- the 680b's world, what MON680 punches and loads.
ASCII text that CARRIES ITS OWN ADDRESSES too, so like HEX it needs no
AT. Data records are S1/S2/S3 (2/3/4-byte addresses), the terminator
S7/S8/S9. Every checksum is verified and a bad one FAILS the load.

S1130100C300F8AF32004D3E0132014D76C9AA5505
S9030000FC
```

```
BIN A flat binary: bytes, and nothing else. It carries NO addresses, so
it cannot say where it goes and AT is REQUIRED. Without one, LOAD
refuses rather than guess.
```

WHICH ONE IS DECIDED BY THE FILE'S CONTENTS, not its name -- Intel HEX and S-records each announce themselves, and a .bin full of HEX text is still HEX. FORMAT= overrides that when the sniff is wrong; it always wins.

AT MEANS 'PUT IT HERE', for both kinds. On a HEX file it relocates: the file's FIRST data record lands at AT and everything else moves by the same amount, wrapping at FFFF (a file whose first record is F000, loaded AT 0, puts its F800 record at 0800). Without AT, a HEX file loads where it says.

ROM is the PROM burner. LOAD writes through the bus, so a ROM never takes it -- a bus write cannot program a PROM on real hardware either, and LOAD says how many bytes landed nowhere rather than

half-loading in silence. ROM goes behind the bus, straight into whichever chip answers reads at that address: the operator pulling the chip and putting it in a programmer. It is why the operator can write a ROM and the guest cannot.

LOAD prog.hex	where the file says, through the bus
LOAD monitor.bin AT E000 ROM	a flat binary, burned into the ROM there
LOAD prog.hex AT 100	relocate: first record goes to 0100
LOAD odd.txt AT 0 FORMAT=HEX	it IS hex, whatever it is called
LOAD mon.txt FORMAT=SREC	read it as Motorola S-records

MOVE — MOV[E]

```
MOVE <range> <dest> [ROM]
```

Copy a range of memory to . It reads the WHOLE range before it writes, so source and dest may overlap either way without a block eating its own tail. The writes are real bus cycles; ROM burns instead, the way EDIT and DEPOSIT do.

MOVE 100-1FF 200	page 1 up to page 2
MOVE 0-FFF 1000	the first 4K, up by 4K

REGS — RE[GS]

```
REGS | SET REG <r>=<v>
```

The flags are registers too, so SET REG C=1 works. A register value is on the wire, so it is HEX.

What it shows is the ACTIVE CPU's own set: the 6800 prints A, B, X, SP, PC and its flags H I N Z V C on one line. The 6809 prints A, B, X, Y, U, S, DP, PC and E F H I N Z V C. Its D (A:B) is reachable by name. CC, the whole condition-code byte, is reachable by name though it is off the line. SET REG takes any name REGS knows -- and only those. BREAK ... IF reads the very same names.

```
REGS
SET REG A=3F
SET REG PC=E000
SET REG X=8000
```

SAVE — SA[VE]

```
SAVE <file> <range> [FORMAT=BIN|HEX|OCTAL|PRN]
```

Memory to a file, through the bus -- so what you get is what the CPU would read, ROM included. The range is what to save; a byte nobody drives reads FF.

THE NAME DECIDES THE FORMAT, and this is the other half of LOAD's rule rather than the same one: LOAD can open the file and see what it IS, and SAVE cannot -- the file does not exist yet. So a name ending .HEX writes Intel HEX, .OCT an octal listing, .PRN (or .LST) a disassembly listing, and anything else a flat binary. FORMAT= says it outright when the name would guess wrong, and wins.

OCTAL and PRN are LISTINGS, not load formats: OCTAL is split-octal addresses and octal bytes, a compact way period listings showed memory; PRN is the DISASM listing -- address, object bytes, mnemonic and any SYMBOLS labels -- written to a file for reading and marking up. Both follow the console base. LOAD does not read either back: BIN and HEX round-trip, OCTAL and PRN do not.

SAVE out.hex 0-FFF	Intel HEX, by its name
SAVE out.bin F800-FFFF	a flat binary, by its name
SAVE out.oct 100-1FF	an octal listing, by its name
SAVE out.prn 100-1FF	a disassembly listing, by its name
SAVE out.dat 0-FFF FORMAT=HEX	hex, though it is not called .hex

SEARCH — SEA[RCH]

```
SEARCH <range> <bytes...>|"str"
```

```
SEA 0-FFFF 7E
SEA 0-FFFF "FLEX"
```

Debugging and tracing

BREAK — B[REAK]

```
BREAK [<addr> | MEM R|W <addr> | TAPE STOP] [IF <expr> | LOADS <expr>] [TRACE ON|OFF]
```

Bare BREAK lists them. Only the first kind is about the CPU at all -- MEM watches BUS CYCLES, so it works unchanged on any processor; TAPE STOP watches a DEVICE, halting when a cassette deck reaches its auto-stop mark -- the way to stop right after a load lands without knowing where the loader ends.

```
BREAK FF13      stop when PC gets there
BREAK 2C00-2CFF ...anywhere in a range
BREAK MEM W 100 stop when anything WRITES 0100
BREAK MEM R 8004 stop on a read of the ACIA data port
BREAK TAPE STOP stop when a cassette deck auto-stops after a load
```

A breakpoint may carry a CONDITION and stop only when it holds. IF tests the registers. A bare word that names a register IS that register, so a literal is written with a leading zero (0A is ten, A is the accumulator). == != < > <= >= compare; && || combine; & | mask.

IF works on EVERY kind. On a plain BREAK it is the PC-arrival state. On a MEM breakpoint it is judged at the instruction BOUNDARY, against the state the instruction began with -- its inputs -- so a conditional cycle breakpoint stops just AFTER the access, where an unconditional one stops just before it.

BREAK MEM R also takes LOADS : like IF, but judged AFTER the read retires, so it sees the byte the read just delivered. IF gates on the inputs; LOADS on the value read.

The names are the ACTIVE CPU's own -- exactly the set REGS shows, every register and flag in it. On a 6800 that is A, B, X, SP, PC and the flags H I N Z V C. On a 6809 it is A, B, D, X, Y, U, S, DP, PC and the flags E F H I N Z V C. A name the running CPU does not have is an error.

```
BREAK 100 IF A==0
BREAK 100 IF X==8000 && Z==1
BREAK 100 IF (A&0F)==0
BREAK MEM R 8004 IF B==5    stop on a read of 8004, but only while B==5
BREAK MEM R 8005 LOADS A>7F ...only when 8005 hands back a byte over 7F
BREAK 100 IF C==1          stop at 100 only when the carry flag is set
```

TRACE ON|OFF makes it a TRACEPOINT: instead of stopping, it turns TRACE on or off and the machine RUNS ON. Two of them trace a REGION and nothing else -- which is how you trace one subroutine out of a program that would otherwise bury you. Like IF, it works on the MEM and IO kinds too. TRACE ON at an address traces the instruction AT it; TRACE OFF does not -- the region is [on, off).

```
BREAK 2C00 TRACE ON      start tracing when PC gets to 2C00
BREAK 2C40 TRACE OFF     ...and stop again at 2C40
BREAK MEM W 2000 TRACE ON start when anything writes 2000 (that write is
                        the first line -- a trace shows its own reason)
BREAK 200 IF X==8000 TRACE ON conditional, and still does not stop
```

Where the trace GOES is TRACE's business, not the tracepoint's: set it up with TRACE ON MASK=..., then TRACE OFF to arm it without emitting. An unconfigured tracepoint traces to the console.

DISASM — DI[SASM]

```
DISASM [<addr>|<range>] [n] [CPU=6800|6809]
```

It needs an INSTRUCTION SET, not a CPU -- so it works on an empty backplane. You normally never type CPU=: the active core says what it speaks, and DISASM asks it. It PEEKS, so it cannot consume a byte from a UART in the range.

n is how many INSTRUCTIONS to decode -- a count, so it is decimal, and 16 when you leave it off. It only applies to a start address: give a RANGE and the range decides where to stop. CPU= names an instruction set, 6800 or 6809. Use it only when the machine has no CPU to ask.

```
DI E000      sixteen instructions of the boot ROM
DI E000 40    forty of them instead
DI           carry on from there
DI 0-2F       exactly that range
DI E000 CPU=6800  decode as 6800 when nothing in the machine can say
```

HISTORY — H[ISTORY]

```
HISTORY [BUS|CPU] [n]
```

The last n INSTRUCTIONS the machine ran, oldest first -- a flight recorder that is always running while the machine runs, so it already holds the run-up to a breakpoint or a crash when you ask. Each line is exactly what STEP prints: the registers and flags as the machine stood, and the decoded mnemonic it was about to run. n is a count, so it is decimal; bare HISTORY shows the last 16.

The mnemonic is decoded from the bytes that ACTUALLY ran at that address, not from what the address holds by the time you look -- so code that rewrote itself (a DDT breakpoint going back, an overlay) reads truthfully. The line reflects the CPU that is in the socket now: on the twin-core card, switching cores makes earlier lines read in the new core's terms.

HISTORY BUS is the other recorder -- the raw BUS CYCLES, no registers and no mnemonics: CYCLE, TYPE (MR/MW a memory read/write), ADDR, DATA, and then the two that make it a BUS trace rather than a CPU one -- who DROVE the cycle (the CPU) and who ANSWERED it. HISTORY CPU names the default out loud.

Each recorder is a FIXED ring of its last 8192: it overwrites its own oldest and never grows, so it costs the same whether the machine ran for a second or a week. Ask for more than 8192 and you get the 8192 it

holds.

```
HISTORY          the last 16 instructions
HISTORY 100      the last hundred instructions
HISTORY BUS      the last 16 bus cycles
HISTORY BUS 100  the last hundred cycles
```

NOBREAK — NO[BREAK]

```
NOBREAK [id]
```

Bare NOBREAK clears them all. An id is not on the wire, so it is decimal.

```
NOBREAK 2
NOBREAK
```

SYMBOLS — SY[MBOLS]

```
SYMBOLS LOAD <file> [REPLACE] | SYMBOLS CLEAR
```

Load an assembler's symbols so you can BREAK, DUMP and EXAMINE by NAME instead of by hex, and SHOW SYMBOLS to read the table.

The file is a Motorola assembler LISTING (.LST or .PRN) -- what as0/as9 write with -l, and the shape the ROM listings in the tree share: a hex address column beside each line, a label heading the source column, a * for a comment. A label defined by EQU is a constant; any other label is a real address -- and only real addresses read back as a NAME: line in a disassembly.

LOAD merges (the newest of a clashing name wins, and it says how many); REPLACE clears first; CLEAR empties the table. A file named in a machine's startup is reloaded on CONFIG LOAD and round-trips through CONFIG SAVE.

```
SYMBOLS LOAD HELLO.LST
SYMBOLS LOAD roms/SWIMON/SWIMON.LST
SYMBOLS CLEAR
```

TRACE — T[RACE]

```
TRACE ON|OFF [file] [MASK=IRQ,CONTENTION]
```

Log every BUS CYCLE while the machine runs -- to the console, or to a file. A cycle, not an instruction: MR/MW are the memory read/write on the backplane, and a cycle two boards both answered is tagged [CONTENTION]. This watches the same stream every board sees, so it is not a CPU feature and works unchanged on any processor.

MASK keeps only the cycles you name (no MASK keeps all): IRQ, CONTENTION. A cycle is kept if it is any of them.

```
TRACE ON          every cycle, to the console
TRACE ON run.log   ...to a file
TRACE ON MASK=CONTENTION just the cycles more than one board answered
TRACE OFF
```

TRACE OFF stops the tracing but REMEMBERS where it was going -- a later TRACE ON, or a tracepoint, resumes to the same file and mask. That is what lets you aim a tracepoint at a file: TRACE ON run.log MASK=CONTENTION, then TRACE OFF to arm it without emitting, then BREAK TRACE ON. See BREAK.

WHO — W[H0]

```
WHO <addr>
```

Who WOULD answer -- it looks without running a cycle, so nothing is consumed and no board is poked. Reports contention.

```
WHO FF00
WHO 8004
```

Configuring the machine

BOARDS — B0[ARDS]

```
BOARDS [LIST]|ADD <type> <id> [k=v...]|REMOVE <id>
```

The backplane: what is in it, what each board answers to, and what is in its sockets. A bare BOARDS lists them. RAM and ROM are named separately, and a ROM range says which image is in it -- an empty socket decodes nothing, so it is not in the memory column at all; it is in UNITS, marked (empty).

```
BOARDS          the backplane
BOARD           the same thing: a prefix of BOARDS
BOARDS ADD memory mem0  fit one -- SHOW BOARDS lists the types
BOARDS REMOVE mem0     pull one out
```

CONFIG — C[ONFIG]

```
CONFIG LOAD <f.toml> | CONFIG SAVE <f.toml>
```

THE MACHINE, NOT WHAT IT IS DOING. SAVE writes the hardware you are actually running -- which boards, in what order, every property SET can write, what each unit is CONNECTed to, what is MOUNTed in each socket, and the startup list. It is the same format you would write by hand, and the same one a built-in is written in, so a saved machine is a first-class machine: LOAD it back, or name it on the command line, and you get exactly what you saved.

IT DOES NOT SAVE STATE, and that is not a gap to be filled: a machine file describes hardware, and none of this is hardware. NOT saved --

```
RAM             what you DEPOSITed is gone. LOAD/SAVE <file> <range> is for
                memory, and it is a separate file for a reason.
the registers    PC included, so a LOADED machine has not started.
breakpoints      nor tracepoints, nor where TRACE was pointed.
CONSOLE          stop and the transforms are the HOST's terminal, not a board
                in the backplane. They survive CONFIG LOAD untouched.
```

A SAVE IS A READ: it asks every property for its value and writes to nothing.

LOAD IS THE WHOLE MACHINE, so it REPLACES the one you have: the boards you had are out of the backplane, the new ones are in, and it is powered up and running its startup list -- a file whose startup says RUN comes up running. Naming that same file on the command line does the identical thing; there is one road.

AND IT IS ALL OR NOTHING. The machine is built off to one side first, so a file that will not load -- a key that does not parse, a disk image that is not there -- leaves you exactly where you were. What you do not get back is the machine you REPLACED: there is no undo but the file you saved it to.

```
CONFIG SAVE machines/mine.toml
CONFIG LOAD machines/mine.toml      ...and this is how you get it back
```

CONNECT — **CONN[ECT]**

```
CONNECT <id>:<u> <endpoint>
```

PLUG IN THE OTHER END OF THE CABLE. A unit is a socket on the back of a board -- one of the UIO's ports, say; an ENDPOINT is the thing at the far end of the cable, on the HOST side of the machine. It is not a board, it has no address, and the guest cannot see it: the 6850 clocks bytes the same way whether the wire ends at your terminal, a telnet session, a real RS-232 port, or nothing at all. No board in the machine knows what any of these words mean.

Endpoints: console | null | loopback | scripted | socket:PORT[?banner] | socket:HOST:PORT | telnet:PORT[?banner=off] | telnet:HOST:PORT | serial:DEVICE | in:PATH | out:PATH | terminal[?emulation=vt100&size=80x24] | printer:QUEUE | |FILE | |socket:PORT

```

console      the host's terminal -- the keyboard and screen you are typing at
null         a cable to nowhere: writes vanish, reads never yield a byte
loopback     the unit's own transmit wired back to its receive, for testing
scripted     a terminal with a caller in place of a human -- what the MCP tools
              and the test suite type into. No tty need exist.
socket:      PORT alone LISTENS; HOST:PORT CALLS OUT. A RAW pipe -- no echo, no
              protocol -- for a PROGRAM at the far end (another machine, a
              transfer). ?banner greets each caller to a listening port.
telnet:      the same, but speaks the Telnet protocol, so a stock `telnet` client
              gets the terminal-server handshake: no double echo, keys sent one at a
              time. Use it when a PERSON telnets in. A listening port greets each
              caller with the machine, line and port; ?banner=off stops it.
serial:      a real port on this host. It is opened at 9600 8N1 and then
              immediately re-programmed by the board, which is the only thing that
              knows what it is strapped to.
in:          PATH -- a host file as a READER (a paper-tape reader): its bytes
              feed the line. ?cps=N (or ?baud=N) paces it -- in:tape.tap?cps=300
              is a slow reader; no option means full speed.
out:         PATH -- a host file as a PUNCH: the line's bytes land on disk,
              8-bit clean, from position 0 and never truncating. Combine them
              for a bidirectional line: in:TAPE.TAP,out:TAPE.PUN
terminal     a windowed terminal the simulator draws itself, no telnet client
              needed. ?emulation=vt100|adm3a|vt52|h19 picks the dialect (vt100
              default), ?size=COLSxROWS the geometry (80x24 default), ?phosphor=
              green|amber the tube colour, ?width=PX the opening window width.
              Needs a window, so only in an SDL build -- headless refuses it.
printer:     QUEUE -- a real print queue on this host (only where the build found
              one). The bytes buffer into a JOB, submitted after a few idle seconds
              (?idle=N, 0=never), on a form feed (?onff), or at a byte ceiling
              (?max=N). 8-bit clean -- a printer control language is not text.
<endpoint>|FILE  a TAP: append |FILE to ANY endpoint above to also log the line,
              both directions, to a hex FILE -- a poor man's protocol analyzer. The
              guest cannot tell it is there. ?fmt=dump|cols|jsonl picks the layout,
              ?ts=elapsed|wall|none the timestamps, ?pins=off drops the modem edges.
<endpoint>|socket:PORT  a live MIRROR: append |socket:PORT to ANY endpoint above
              and a second person can `telnet localhost PORT` to WATCH the session --
              and TYPE, sharing the line (take-over). ?ro makes it watch-only. The
              watcher never paces the guest; a slow one loses scrollbar, not a byte.

```

Exactly ONE unit may hold the console; connecting a second STEALS it and says who from. Two boards reading one keyboard would each get half the characters.

```
CONN mps0:tty console
CONN uio0:serial null
CONN uio0:serial loopback
CONN uio0:serial socket:2323      `telnet localhost 2323` now reaches the guest
CONN uio0:serial telnet:2323      ...the same, but no double echo for a human
CONN uio0:serial socket:bbs.example:23  the guest dials OUT, to somebody else's port
CONN uio0:serial serial:/dev/tty.usbserial-AL009KFH  a real cable, real hardware
CONN uio0:serial serial:COM3          ...the same, on Windows
CONN uio0:plb out:printout.txt        capture a printer to a file
CONN uio0:pla in:TAPE.TAP?cps=300     a paper-tape reader
CONN uio0:plb out:TAPE.PUN           a paper-tape punch
CONN mps0:tty terminal?emulation=adm3a  a windowed ADM-3A of its own
CONN uio0:plb printer:linewriter      print to a real host queue
CONN uio0:serial socket:2323|bbs.hex?fmt=cols  telnet in, and TAP it to a log
```

DISCONNECT takes the cable out again; SHOW CONSOLE says which unit holds it.

CONSOLE — CONS[OLE]

```
CONSOLE [<k>=<v>...]
```

The host's terminal -- your keyboard and screen -- and the knobs that shape how bytes cross it. Bare CONSOLE prints those settings (and which board unit is wired to the terminal); CONSOLE k=v changes one. It is pure shorthand: CONSOLE alone does what SHOW CONSOLE does, and CONSOLE k=v does what SET CONSOLE k=v does -- the same two commands, spelled short.

The keys k, and the value v each takes:

```
stop      a control byte 01-1F (HEX): the key that returns to the monitor (alias: attn)
base      hex | octal -- the operator's number base for what it PRINTS
history   lines saved in .swtpcsim_history in the launch dir (default 50; 0 = off)
upper     on|off: fold typed input to uppercase (much period software insists)
strip7in  on|off: mask the high bit on input
strip7out on|off: mask the high bit on output (MITS BASIC's end-of-message)
crlf      on|off: add LF after every CR the guest prints -- usually WRONG
echo      on|off: local echo, for half-duplex hardware
bell      on|off: pass 07 through to the host bell
bsdel     off | bs (fold DEL->BS) | del (fold BS->DEL)
log       a host file to copy the whole session to -- guest output and the
          keys you type both; off (or an empty path) stops it
```

These are the TERMINAL's, not a board's; a board's own line coding (baud, data_bits) is SHOW tty. SHOW CONSOLE lists these with their current values.

SET CONSOLE DEBUG= is not one of these: it aims where the machine's diagnostic channels print (SHOW DEBUG), not the terminal, so SHOW CONSOLE does not list it. See SET ... DEBUG.

STOP is the key that takes the keyboard BACK from a running guest. The host intercepts it before the guest is ever offered the byte, so the guest cannot disable it -- and that is why it must not be a key the guest needs.

```
CONSOLE          the settings, and which board unit is wired to the terminal
CONSOLE stop=1D   make it ^] (hex: it is a byte on the wire; attn= also works)
CONSOLE upper=on strip7out=on  two at once, the classic MITS BASIC pair
```

This command does NOT choose which board is the console -- CONNECT does that (CONNECT : console); bare CONSOLE only reports the one now wired.

DISCONNECT — DISC[ONNECT]

```
DISCONNECT <id>:<u>
```

The line then goes nowhere. NOT an error: an unconnected 6850 sits there with TDRE set forever, and a program that writes to it works fine and talks to nobody -- which is exactly what the card does with no cable in it.

```
DISC uio0:serial
```

DO — D0

```
D0 <file>
```

Run a FILE of monitor commands, one per line, as if you had typed each here. A DO file is a machine's startup list living in a plain text file -- the config language and the command language are one language, so anything you can type, a DO file can do.

PATHS INSIDE IT ARE RELATIVE TO THE FILE, not to where you are standing -- so D0 examples/flex/boot.ini mounts the disk beside that file from anywhere. Blank lines and ; or # comments are skipped, so a DO file reads like a script.

It runs against whatever machine is loaded, so a DO file usually opens with MACHINE to pick its own base -- MACHINE swtpc then MOUNT/RUN, or MACHINE none then BOARDS ADD to build one from scratch. On

the command line, `swtpcsim -s FILE` runs the same file at startup and exits with its status. FILE is named from where you launched, and the paths in it are relative to it, as in a DO file.

It is a LINE RUNNER, not a full scripting language: no arguments, no IF or GOTO. For conditional or interactive automation, drive a live guest over `--mcp`.

```
DO flex.ini                paths relative to flex.ini's own directory
DO examples/flex/boot.ini  run it from anywhere
```

MACHINE — MA[CHINE]

```
MACHINE <name> | MACHINE none
```

Load a BUILT-IN machine by name, replacing whatever is in the backplane. It is the runtime twin of naming one on the command line (`swtpcsim swtpc`) and the command form of a machine file's `base = "<name>"` -- so a DO script opens with it and no longer depends on how it was launched. SHOW MACHINES lists the names; SHOW MACHINE shows what is in one.

MACHINE none is the empty backplane `-n` gives you -- no boards, nothing driving anything -- where you START a machine you build up by hand with BOARDS ADD.

It powers the result (RAM filled, ROM images read, POC* pulsed) but does NOT run the built-in's startup: like `base =`, it gives you the HARDWARE, and the lines after it do the MOUNTing and RUNning. For a machine from a FILE, use CONFIG LOAD.

```
MACHINE swtpc                a SWTPC 6800, SWTBUG in ROM at E000
MACHINE none ; BOARDS ADD 6800 cpu0 ; ... ; POWER ; RUN E000
```

MOUNT — M[OUNT]

```
MOUNT <id>[:<u>] <file> [WP] [CREATE] [extract[=<base>]] [k=v...]
```

Put a disk in a drive, a tape in a recorder, or an image in a ROM socket. WP write-protects it: the guest may read it and may not write it. RO is accepted and means the same -- it is the word for a ROM, which is read-only because of what it is.

CREATE makes the file first if it is not there (empty), then mounts it -- a fresh hard-sector disk to FORMAT, or a blank cassette. Without CREATE a missing file is a 'no such file', because a mistyped name is a mistake, not a new disk.

A TRAILING `k=v` SETS A UNIT PROPERTY, applied the moment the medium is in -- the same properties SHOW lists and SET : writes, said at the one moment you were going to say them anyway. A unit with no

such property says so rather than ignoring you.

EXTRACT is not a property and runs after the mount: it splits a cassette WAV into one .TAP per program on it, exactly as the EXTRACT verb does. extract= names those files instead of taking the default.

A NAME IS CASE-BLIND, and you may leave off what carries no information: the trailing index when only one such board is in the machine, and the unit when the board has only one you could mount into. Anything genuinely plural you must say, and it will tell you so.

```
MOUNT dc40:drive0 disks/flex.dsk
MOUNT dc40:drive1 disks/master.dsk WP
MOUNT dc40:drive1 new.dsk CREATE    a blank disk to FORMAT from the guest
MOUNT mem0:rom0 roms/monitor.bin
MOUNT ACR tape.bin    the one cassette, its one tape: acr0:tape
MOUNT ACR new.wav CREATE mode=record a blank tape, in and recording
MOUNT acr0:tape program.wav extract  mount a WAV and split it into .TAP files
```

SHOW MOUNTS is the other half of this command: every socket in the machine, what is in it, and which are still empty. UNMOUNT takes it back out. A relative path resolves against the machine's own directory - the same folder the machine file's own mounts come from -- as SHOW PATHS describes.

REGION — REGI[ON]

```
REGION ADD <id> type=ram|rom at=<addr> [size=|mount=]
```

A region is a POPULATED part of a board. What is not covered by one is an empty socket: it decodes nothing and floats to FF. **at** is an address, so it is hex; **size** is a size, so it is decimal, and K/M work.

```
REGI ADD mem0 type=ram at=0 size=48K
REGI ADD mem0 type=rom at=E000 mount=builtin:swtbug
```

RESTORE — REST[ORE]

```
RESTORE <file>
```

Load a SNAPSHOT back into THIS machine. The machine must be the same shape the snapshot was taken from -- the same boards, same ids, same order (build it with the same machine file, or a CONFIG LOAD, first) -- and a file that does not match is refused with the reason, the running machine untouched.

```
RESTORE before-boot.snap
```

SET — SE[T]

```
SET <id>[:<u>]|CONSOLE|DISPLAY|MACHINE|REG|BUS <k>=<v>
```

Each property has a base of its own -- an address is hex, a baud rate is decimal. SHOW lists them all, with each value.

A UNIT HAS PROPERTIES OF ITS OWN, and : is how you reach them: the tape in the recorder rather than the recorder, the disk in the drive rather than the controller. SHOW prints both tables, the board's and each unit's.

CONSOLE and DISPLAY are the HOST's terminal and video window rather than boards, and they take settings the same way. REG is a CPU register (see REGS), and BUS is the backplane's own diagnostics rather than anything plugged into it. MACHINE is the machine itself: its name is what SHOW MACHINE prints, the video window's title, and what CONFIG SAVE writes.

```
SET mem0 fill=zero
SET kc0:tape mode=record    the tape in the recorder, not the recorder
SET vdm0 width=1024        how wide the video window opens, in pixels (auto = ~half the screen)
SET DISPLAY focus=on       the video window takes the keyboard, not the terminal
SET DISPLAY crt=on         paint the window like the period tube: soft phosphor and 4:3
SET MACHINE name=mybox     what CONFIG SAVE calls the machine
SET REG A=3F               a register in the CPU that is in the socket
SET BUS UNCLAIMED=WARN     warn on a cycle no board answered
                           (also CONTENTION=WARN|ERROR|SILENT, UNCLAIMED=WARN|HALT|SILENT)
```

SHOW — SH[OW]

```
SHOW <id>|BOARDS|BOARD <type> [UNITS]|MACHINES|MACHINE [<name>]|BUS
[MAP|IRQ|CONTENTION]|ROMS|MOUNTS|PATHS|CONSOLE|DISPLAY|SYMBOLS|CLOCK|VERSION
```

```

SHOW mem0      regions and properties
SHOW BOARDS    the board types you can add
SHOW BOARD dc4 one type's description and properties (add UNITS for just those)
SHOW MACHINES  the built-in machines you can boot
SHOW MACHINE   the current machine (add a name for a built-in's detail)
SHOW BUS MAP   who decodes what, and what floats
SHOW BUS IRQ   the IRQ and NMI wires, who pulls them, and the vectors at FFF8-FFFF
SHOW MOUNTS    every disk, tape and ROM in the machine, and what is in it
SHOW PATHS     what a path resolves against -- and there is more than one answer
SHOW CONSOLE   which unit holds the keyboard, and its transforms
SHOW DISPLAY   the host video window: keyboard focus, and the CRT look
SHOW TERMINAL  the built-in terminal's transforms (strip7out, cr, bsdel, ...)
SHOW SYMBOLS   the loaded symbols (SHOW SYMBOLS SIO* filters); load them with SYMBOLS
SHOW CLOCK     emulated time: cycles since POWER, and what they are in seconds
SHOW ROMS      the ROM images built into this binary, and where each came from
SHOW VERSION   which build this is, and the commit it was built from

```

SNAPSHOT — SN[APSHOT]

```
SNAPSHOT <file>
```

Write the machine's STATE to a file: the CPU, the clock, and every board's registers, RAM and latches. NOT its configuration -- a snapshot is state, the way a machine file is configuration. RESTORE reads it back.

```
SNAPSHOT before-boot.snap
```

STARTUP — STA[RTUP]

```
STARTUP [ADD <command> | REMOVE <n> | CLEAR]
```

The machine's boot list -- the commands a config replays on load, and what CONFIG SAVE writes out as startup = [...]. A bare STARTUP shows the list, numbered; the rest edit it in place, so you can compose a boot sequence at the prompt and save it:

```

STARTUP          show the list, numbered
STARTUP ADD MOUNT dc40:drive0 "FLEX2-40.DSK"  append a line, verbatim
STARTUP REMOVE 2  drop line 2
STARTUP CLEAR     empty the list

```

ADD takes the REST OF THE LINE exactly as typed -- quotes, spaces and all -- because a startup entry is just a command line: anything valid at the prompt is valid in the list, so it is stored unchecked, the same as a line you write in the file by hand.

```
STA ADD RUN FF00
```

UNMOUNT — U[NMOUNT]

```
UNMOUNT <id>:<u>
```

Takes the disk, tape or ROM out of the unit. A drive or a tape recorder is then empty. A ROM socket is then empty too: those pages float to FF, as on a board with no chip in the socket.

```
U dc40:drive0
```

Getting help and leaving

HELP — HE[LP]

```
HELP [<command>]
```

Bare HELP lists the commands and nothing else -- the whole set on a few lines, which is what you want when you are hunting for the name. HELP with a command gives the usage and the examples.

```
HELP          the list
HELP DUMP     the detail
?             the same as HELP
```

QUIT — Q[UIT]

```
QUIT
```

Leave the monitor and end the program. It does NOT ask: the machine lives in memory, so anything you have not written out -- CONFIG SAVE, SAVE, SNAPSHOT -- is gone with it. There is no EXIT; QUIT is the one word.

```
QUIT
```